

Domain-Driven Design -- DDD Basics

Entity . Value Object . Aggregate . Repository . Domain Event . Bounded Context

Sub-Chapter 02-09 | Simplest Examples

Entity	Has a unique identity that persists over time
Value Object	Defined by its value - no identity, immutable
Aggregate	Cluster of objects treated as one unit
Repository	Gateway to retrieve and save aggregates
Domain Event	Something important that happened in the domain
Bounded Context	Clear boundary around a sub-domain and its model

1 Entity

Has a unique identity that persists over time

An Entity has a unique ID. Even if its data changes, it is still the same object as long as the ID is the same. Example: a User. Alice can change her name, but she is still Alice (same ID).

```
class User {
    private final int    id;        // identity -- never changes
    private              String name; // data -- can change

    User(int id, String name) { this.id = id; this.name = name; }
    int    getId()    { return id; }
    String getName() { return name; }
    void    setName(String name) { this.name = name; }

    // Equality based on ID, not data
    public boolean equals(Object o) {
        if (!(o instanceof User)) return false;
        return this.id == ((User) o).id;
    }
    public String toString() { return "User{id="+id+", name="+name+"}"; }
}

// Usage
User u1 = new User(1, "Alice");
User u2 = new User(1, "Alice Smith"); // same user, name changed
System.out.println(u1.equals(u2));    // true  -- same ID
User u3 = new User(2, "Alice");       // different user
System.out.println(u1.equals(u3));    // false -- different ID
```

Entity Diagram

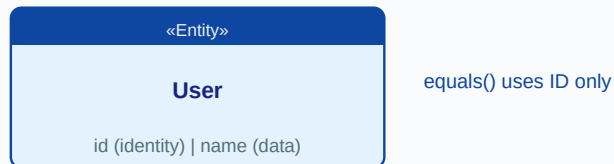


Figure 1 -- Entity: equality based on ID

2 Value Object

Defined by its value - no identity, immutable

A Value Object has no identity. Two Money objects of Rs 100 are completely interchangeable. They are immutable -- once created, never changed.

```
final class Money {
    private final int    amount;
    private final String currency;

    Money(int amount, String currency) {
        this.amount    = amount;
        this.currency = currency;
    }
    // No setters -- immutable!
    Money add(Money other) {
        if (!this.currency.equals(other.currency))
            throw new IllegalArgumentException("Currency mismatch");
        return new Money(this.amount + other.amount, this.currency);
    }
    public boolean equals(Object o) {
        if (!(o instanceof Money)) return false;
        Money m = (Money) o;
        return amount == m.amount && currency.equals(m.currency);
    }
    public String toString() { return currency + " " + amount; }
}
// Usage
Money price = new Money(100, "INR");
Money tax   = new Money(18,  "INR");
Money total = price.add(tax);
System.out.println(total);                // INR 118
System.out.println(price.equals(new Money(100,"INR"))); // true -- same VALUE
```

Value Object Diagram

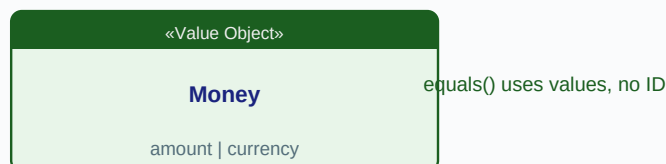


Figure 2 -- Value Object: equality based on all fields

3 Aggregate

Cluster of objects treated as one unit

An Aggregate is a group of related objects. One object is the root -- all access goes through it. Example: Order is the root; it owns OrderItems. You never modify an OrderItem directly -- always go through Order.

```
class OrderItem {
    String product; int qty; double price;
    OrderItem(String product, int qty, double price) {
        this.product=product; this.qty=qty; this.price=price;
    }
    double subtotal() { return qty * price; }
}
// Order = Aggregate Root
class Order {
    private int id;
    private List items = new ArrayList();

    Order(int id) { this.id = id; }

    void addItem(String product, int qty, double price) {
        items.add(new OrderItem(product, qty, price));
    }
    double total() {
        return items.stream().mapToDouble(i -> i.subtotal()).sum();
    }
    public String toString() { return "Order#" + id + " total=" + total(); }
}
// All access through the aggregate root
Order order = new Order(1);
order.addItem("Book", 2, 200.0);
order.addItem("Pen", 5, 10.0);
System.out.println(order); // Order#1 total=450.0
```

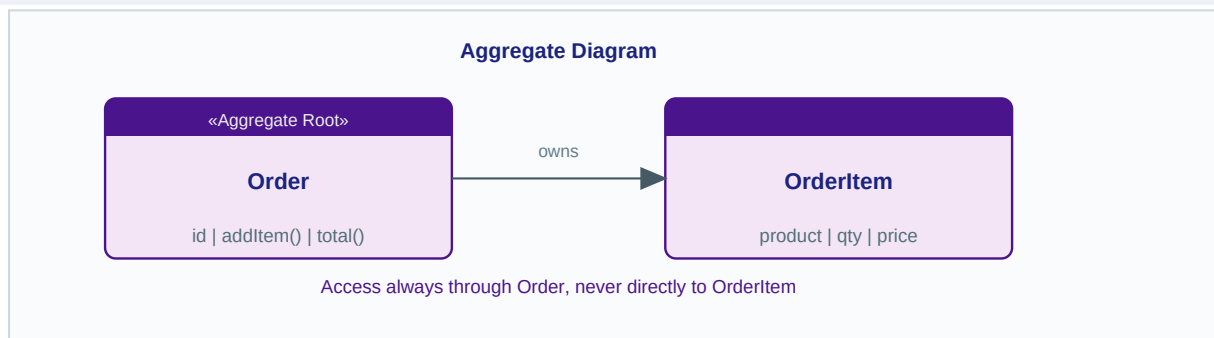


Figure 3 -- Aggregate: access only via root

4 Repository

Gateway to retrieve and save aggregates

Repository hides the storage details. Your domain code just calls `repo.save()` and `repo.findById()` -- it does not know or care if data is in a DB, file, or memory.

```
interface OrderRepository {
    void save(Order order);
    Order findById(int id);
}

class InMemoryOrderRepository implements OrderRepository {
    private Map store = new HashMap();
    public void save(Order o) { store.put(o.getId(), o); }
    public Order findById(int id) { return (Order) store.get(id); }
}

// Domain code never knows about storage details
OrderRepository repo = new InMemoryOrderRepository();
Order o = new Order(1);
o.addItem("Laptop", 1, 80000.0);
repo.save(o);
Order found = repo.findById(1);
System.out.println(found); // Order#1 total=80000.0
```

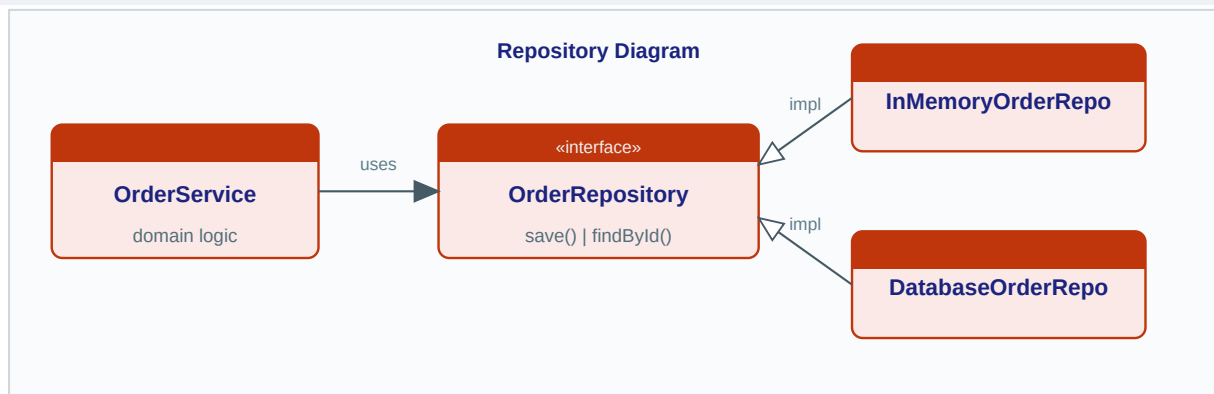


Figure 4 -- Repository: interface hides storage

5 Domain Event

Something important that happened in the domain

When something important happens in the domain, publish an event. Other parts of the system react to it independently -- loose coupling.

```
// Event -- plain data object
class OrderPlacedEvent {
    int orderId; String customerName;
    OrderPlacedEvent(int orderId, String customerName) {
        this.orderId=orderId; this.customerName=customerName;
    }
}
// Listener
interface EventListener { void onEvent(OrderPlacedEvent event); }
class EmailNotifier implements EventListener {
    public void onEvent(OrderPlacedEvent e) {
        System.out.println("Email sent to " + e.customerName +
            " for order #" + e.orderId);
    }
}
// Simple event bus
class EventBus {
    private List listeners = new ArrayList();
    void subscribe(EventListener l) { listeners.add(l); }
    void publish(OrderPlacedEvent e) {
        listeners.forEach(l -> l.onEvent(e));
    }
}
// Usage
EventBus bus = new EventBus();
bus.subscribe(new EmailNotifier());
bus.publish(new OrderPlacedEvent(42, "Alice"));
// Email sent to Alice for order #42
```

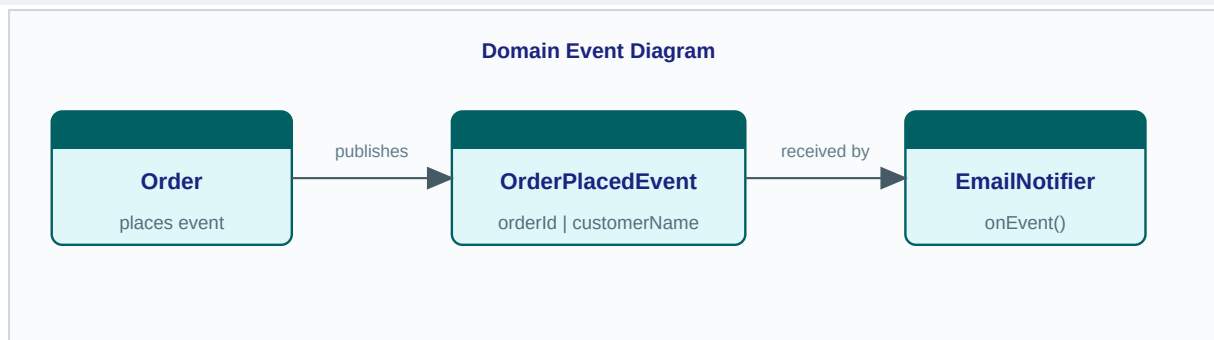


Figure 5 -- Domain Event: loose coupling via events

6 Bounded Context

Clear boundary around a sub-domain and its model

In a big system, the word "Customer" means different things in different sub-domains. Bounded Context gives each sub-domain its own model so they don't interfere.

	Order Context	Shipping Context
"Customer" means	Buyer with payment details	Delivery address holder
"Order" means	Items + prices + payment	Package to be dispatched
Cares about	Payment, discounts	Route, tracking, carrier

```
// In package: com.myapp.ordering
class Customer {
    int id; String name; String creditCard;
}

// In package: com.myapp.shipping
class Customer {
    int id; String name; String deliveryAddress;
}
// Same word "Customer" -- two different models
// They NEVER share the same class -- that would mix concerns
```

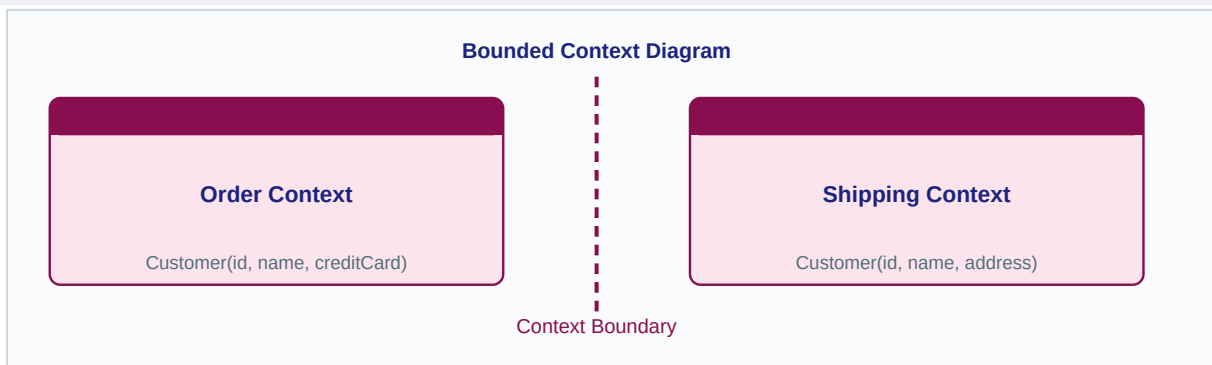


Figure 6 -- Bounded Context: separate models per sub-domain

DDD Cheat Sheet

Quick reference for all six concepts

Concept	What it is	Key rule	Equals based on	Example
Entity	Has unique ID	Same ID = same object	ID	User, Order
Value Object	Defined by value	Immutable, no ID	All fields	Money, Address
Aggregate	Group with one root	Access only via root	N/A	Order + OrderItems
Repository	Storage abstraction	Interface hides DB	N/A	OrderRepository
Domain Event	Something happened	Past tense, immutable	N/A	OrderPlacedEvent
Bounded Context	Sub-domain boundary	Own model per context	N/A	Order vs Shipping